

SQL Injection in WHERE Clause Allowing Retrieval of Hidden Data

1. Vulnerability Overview

- **Vulnerability Type:** SQL Injection — WHERE Clause
- **Security Impact:** Unauthorized Data Access
- **Primary Objective:** Retrieve unreleased products by manipulating the application's product-category filtering mechanism.

2. Vulnerability Description

The application dynamically constructs a database query using user-controlled input within the **WHERE** clause.

Insufficient input handling allows an attacker to manipulate the structure of the underlying SQL query. By injecting SQL syntax and neutralizing part of the original query logic, an attacker can bypass application-level filtering conditions, including the **released = 1** restriction.

As a result, products that are intended to remain hidden or unreleased may become accessible through the application's response.

3. Attack Vector & Exploitation Methodology

The assessment involves identifying a user-controlled parameter that is incorporated directly into the SQL **WHERE** clause.

The attacker then modifies the parameter to alter the intended query logic and bypass the application's product-visibility restriction.

The resulting query may return records that should not be exposed through the normal application workflow.

4. Security Impact

Successful exploitation may allow an unauthorized user to:

- Bypass application-level data filtering.
- Retrieve unreleased or hidden product records.
- Access information outside the intended business logic.
- Potentially expose additional database information depending on the vulnerable query context.

5. Root Cause

The primary root cause is the direct incorporation of user-controlled input into dynamically constructed SQL statements without adequate separation between application data and SQL query logic.

6. Remediation

- Implement **parameterized queries / prepared statements** for all database interactions.

- Ensure user-controlled input is treated strictly as data rather than executable SQL syntax.
- Avoid dynamically concatenating untrusted input into SQL statements.
- Apply appropriate server-side input validation as a secondary security control.
- Restrict database permissions according to the **Principle of Least Privilege**.

7. Security Recommendation

Parameterized database interfaces should be used consistently throughout the application to prevent attackers from modifying the logical structure of backend SQL queries.

Author: Moe Htet Ar Kar (Phoe Cho)